TOAD CITY BALTIMORE · OUTSIDE-AI BRIEF · CLAUDE SONNET

Environment: rowhouse kit, street props and vehicles

Tasks A and B · merged city geometry and rideable vehicles · drop-in three.js modules



Recommended setting. Use **Claude Sonnet** (the newest Sonnet available to you) with **extended thinking ON at medium effort**, and **high effort for the first kit piece** (the Formstone rowhouse), the first prop and the first vehicle, because those set the pattern. Work on one kit piece or prop family per session and self-test it in `viewer.html` before you move on. Hand finished, QA-passed modules to **Claude Haiku** for variations (Appendix B).

Prepared for Brad Parlette, 11 October 2026. Everything is built procedurally in code; real Baltimore signage is redrawn on canvas, never copied from logo files. The starter kit (`toadcity_kit.js`, `viewer.html` and two example modules) comes as `toadcity_starter_kit.zip`. Its core is printed in the appendix in case the zip is missing.

Contents

1 · Project overview 3

2 · Style guide and quality bar 5

3 · Code starter and interface 7

4 · Task A: Baltimore rowhouse and city-block kit 9

5 · Task B: Street props and vehicles 13

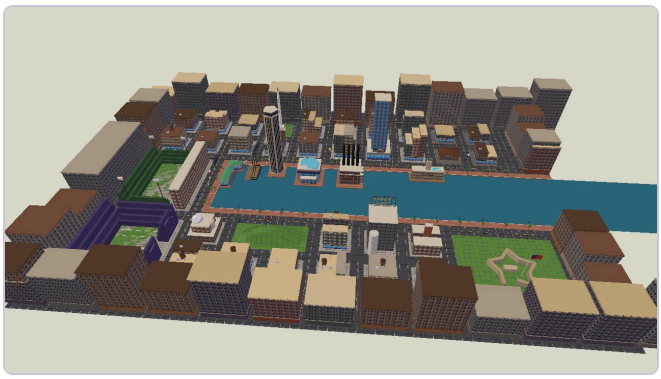
6 · Delivery instructions 16

Appendix A · Kit core (props and city) 17

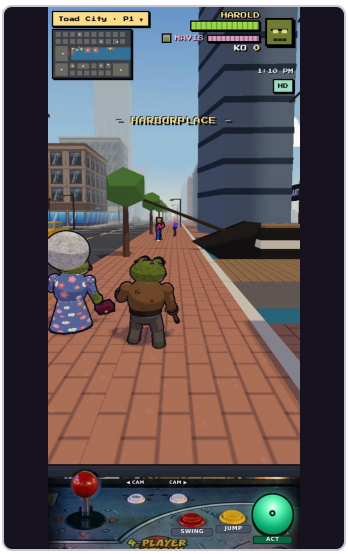
Appendix B · Haiku helper prompts 19

1 · Project overview

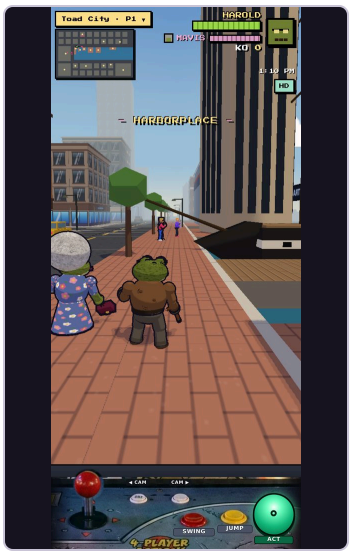
Toad City Baltimore is an open-world, cel-shaded street brawler set on Baltimore's Inner Harbor, built for phones first. You play **Harold**, an elderly toad with a cane. His wife **Mavis** (grey afro, floral dress, a handbag she swings) fights beside you under AI control. Together you clear crime out of each district around the harbour. Each landmark district has a set-piece fight and a boss, and the final boss is Mister Formstone. The real map is compressed about 3×: landmarks sit about one block apart, with a 6 m brick promenade along the water and land about two blocks deep. Day turns to night every 5 minutes (a 600 s loop), and at night the windows, street lamps and neon light up.



City overview, day: the harbour basin, the street grid and the landmark massings (in-house). Night lights about 45% of windows and warm lamps along the promenade.



iPhone portrait 390×844: the game view above the arcade control panel.



The arcade panel (ball-top stick, round buttons, big ACT button). Assets must read in the view above it.

Tech stack

- **three.js r160** with plain ES modules. The game source is a set of `src/*.js` files that a shell script concatenates into one module (`game.js`). There is no bundler and no framework.
- **Everything procedural:** geometry comes from three.js primitives in code, and every texture is drawn on a `<canvas>` at load. There are no image, model or font files to license. Audio is separate (not part of this work).
- **Look:** MeshLambert plus a 3-band toon ramp injected with `onBeforeCompile`, vertex colours, inverted-hull ink outlines and PCF soft shadows from one sun. A `?retro=1` 32-bit mode exists, but the kit is HD-only and that is what you deliver for.
- **How deliveries plug in:** each module is a file that receives the kit object `K` (the same functions the game uses). In-house, we strip the `export` keywords, append the file to `src/`, bind `K` to the game's globals and register the build function in the gang, boss, prop or vehicle tables. If your module runs in the kit viewer, it runs in the game.

Target devices and performance budgets

The main target is an **iPhone in portrait, 390×844 CSS px at DPR 2** (the game view is the tall area above the arcade panel), plus desktop Chrome and Safari. Aim for 60 fps, with a 30 fps floor on a mid-range iPhone. Current city frames measure **76–168 draw calls and 99k–346k triangles** at five spots (day, 390×844, including the shadow pass), so new assets must be lean.

Budget	Limit	Notes
Whole frame (iPhone)	≤ 250 draw calls, ≤ 400k tris	Includes the shadow pass. The in-house team owns this number; your assets must not blow it.
Reference: Harold / Mavis	60 / 54 parts · 14.0k / 15.7k tris	Measured in the kit viewer. Each part is a mesh plus an ink hull.
Reference: current thug	26–33 parts · 7.6k–8.5k tris	Two materials (one toon plus the shared outline).
New gang member (Task C)	≤ 40 parts · ≤ 12k tris · ≤ 3 mats	Far LOD (`opts.lod = 1`): ≤ 14 parts, ≤ 3k tris, no fine detail.
Boss (Task D)	≤ 70 parts · ≤ 22k tris · ≤ 4 mats	One boss on screen at a time.
Rowhouse kit (Task A)	house ≤ 2.5k · corner ≤ 3.5k · block ≤ 30k tris	Merged: a full block is ≤ 4 draw calls. One 1024² atlas for the whole kit.
Static props (Task B)	120–2,500 tris each	Merged into the chunk, so they add no draw calls. Per-prop limits are in the task.
Vehicles (Task B)	600–6,000 tris · 3–12 meshes	Their own Group, so they move. At most 2 materials each.
Textures	canvas only · ≤ 1024² atlas · ≤ 256² per character map	Drawn in code at load; mipmapped in HD.

Merged geometry and shared materials: the static city is written into merged builders, each 64 m chunk being one toon mesh, one outline hull, one glow mesh and one decal mesh, and the chunks are frustum-culled. Characters are groups of parts that share one toon material per rig. **LOD the game already does:** thugs beyond 24 m drop their ink hulls and stop casting shadows, and beyond 48 m they sleep (hidden). Hulls must therefore be BackSide ShaderMaterial children so this code can find them.

Coordinates and scale

Item	Value	Item	Value
Units / axes	metres · Y up · X east · Z south (north = −Z)	Facing at yaw 0	+Z (characters, vehicles, house fronts)
City block	24 × 24 m	Street	10 m wide (grid period 34 m)
Promenade	6 m brick strip on every water edge	Kerb / pavement top	`CURB = 0.16`
Ground floor / upper floor	`GF = 4.4` / `FH = 3.4`	Parapet	`PAR = 1.3`
Water surface	`WY = −0.85`	Render chunk / collision cell	64 m / 16 m
World bounds	x −136…213, z −106…104	Harbour basin	x −60…400, z −30…30
Harold / Mavis	about 1.6 m / about 2.0 m (afro)	Thug / bruiser	about 2.0 m / ×1.2

> **Signage rule.** Brad wants the city to look like the real Inner Harbor: storefronts, signs and props **may use real Baltimore businesses' names and sign styles where they really exist** (Phillips Seafood, Barnes & Noble and Hard Rock Cafe at the Power Plant, the water-taxi branding, Natty Boh-style corner-bar signs, Utz, Old Bay). The neon sign in section 2 shows how: the real script wording **redrawn on canvas**, never a copied logo file or photo. **Bosses and thugs stay original characters** (no real people).

Readability on a phone: silhouettes must read at 6–10 m on a 390×844 screen; exaggerate key shapes by 10–20% (heads, hands, cornices, stoops, lamp heads); give each gang, district or vehicle one accent colour; check every asset in both the day and the night preset before delivery.

2 · Style guide and quality bar

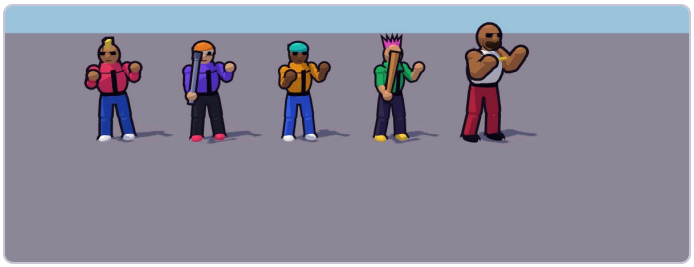
Everything you deliver must look like it belongs next to Harold and Mavis below. These are the **asset starters**: renders of the real game models through the kit viewer you were given. They are the quality bar.



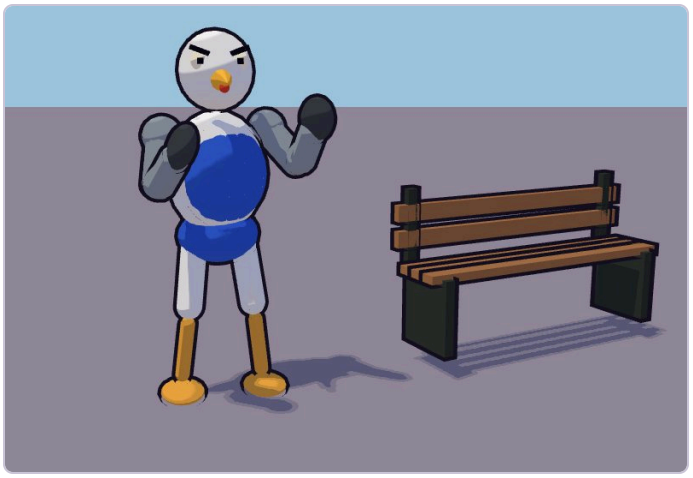
Harold and Mavis, day preset: chunky primitives, warty skin map, floral and curls maps, three hard cel bands, ink hulls.



The same at night: hemisphere and sun drop, a warm lamp and a cool camera-side fill keep the characters readable.



The current thugs (to be replaced by Task C gangs): readable but plain. Your work must reach Harold-level detail.



The kit examples: a gull thug (character interface) and a bench (prop interface, with an atlas decal). The readout shows the budget check.

Palette

#14101f INK: outlines, black	**#6f7f34** Harold skin	**#b8b47a** Harold throat	**#7a5232** Harold cardigan	**#5e3e24** cardigan trim	**#5a5248** Harold slacks	**#7f8f3e** Mavis skin	**#5f73b0** Mavis dress base
#f08ab0 floral pink	**#c8c6c2** Mavis afro	**#6a1f3a** handbag	**#c8a040** gold: clasp, bangle	**#e8d888** toad eye gold	**#ff3d6e** accent pink-red	**#7b4dff** accent violet	**#ff9f1c** accent orange
#23c26b accent green	**#2f5bd3** accent blue	**#ffd23c** accent yellow	**#1fd3c6** accent teal	**#8a4a3a** red brick	**#6f6a62** grey stone	**#9a6a44** tan brick	**#5a4f5c** plum

#4f5a52 green-grey	#a4907a sandstone	#c9b48a trim cream	#eae6dc marble stoop	#ffa050 lamp light	#ffe08a lit window	#ff3a1c neon red	#c8d0ff night camera fill

Use mid-value base colours, because the toon ramp multiplies them by 0.46, 0.78 or 1.06. **Never use pure #000 or #fff**: use INK `#14101f` for black and at most `#f2f2f2` for white. Shadows are tinted slightly cool.

Toon ramp (cel bands)

- The lighting ratio *r* (lit luminance ÷ base luminance) is snapped to three bands: **shadow 0.46** (r below 0.38–0.44), **mid 0.78** and **lit 1.06** (r above 0.86–0.92). Edges are hard (smoothstep over 0.06).
- The shadow band is tinted by (0.88, 0.86, 1.10) at 50%. Strong local light (r above 1.35, such as a lamp at night) passes through unbanded.
- A hit-flash uniform mixes the whole rig to white (`mat.userData.flash`). Every sub-material of a rig must share it.

Ink outlines

- The outlines are **inverted hulls**: back faces of the same geometry pushed out along a mitred vertex direction (`odir`, from `addOdir()`), coloured INK `#14101f`.
- Width = `uW × camera distance`, with the distance clamped to 2.5–36 m. **Characters 0.0042**, **city 0.0034**, **props 0.0030**. That gives a near-constant 2–3 px line on a phone, which fades into the fog colour with distance.
- Only major masses get hulls. Use `outline: false` on parts thinner than about 4 cm (characters) or 6 cm (buildings), and on flat decals. Never use screen-space edge detection.

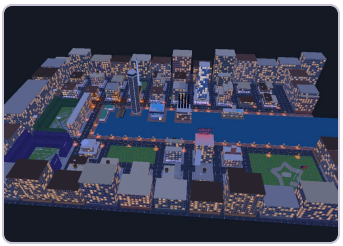
Night lighting rules

Preset	Sky / fog	Hemisphere (sky / ground, intensity)	Sun (colour, intensity, direction)
Day	#9cc4dc / #b4c0c6	#dfe6ff / #8a7f74, 1.45	#fff3dc, 2.1, (−0.5, 1.7, 0.42)
Dusk	#b0706a / #8a6060	#ffb08a / #5a4040, 0.85	#ff9a58, 1.25, (−1, 0.32, −0.25)
Night	#0a0f1a / #0b0f17	#4a5a88 / #14141c, 0.32	moonlight #8aa0e8, 0.45, (0.45, 1.2, −0.9)

- **NIGHT** (0 for day to 1 for night) drives every glow. Windows light by a stable seeded hash (about 45% lit, warm `#ffe08a`/`#ffbf3c`). Shop glass, signs and neon always glow.
- Street lamps are warm orange `#ffa050`: additive glow pools on the ground, billboard halos, and 4 pooled point lights (intensity 16, range 15 m) that follow the hero. Assets **never add their own lights**; they register lamp points instead.
- A cool camera-side fill (`#c8d0ff`, 3.2) keeps characters readable at night. Do not make character bodies emissive; only intentional glow parts (a ghost, crystals, sparks) may glow.



Night street (in-game): lit windows, lamp pools and shop signs (current placeholders; real Baltimore signs are now wanted).


City at night: about 45% of windows lit, warm promenade lamps.



Neon sign quality bar, day: canvas-drawn lettering on a steel lattice.



The same sign at night: red-orange neon, soft glow, water reflection streak.

3 · Code starter and interface

Every delivery is a **drop-in JS module**: no imports, one `META`, build functions that take `K`. Below is a real excerpt from the game, the opening of the thug builder exactly as it ships, followed by the kit examples that define the interface you must follow.

Real excerpt: how the game builds a thug (src/04_actors.js)

```
function buildThug(look) {
  const mat = toonMat(); const d = THUG_D, J = skeleton(d, mat);
  const { jacket, pants, skin } = look;
  part(J.hips, G.sph(0.22, 0.15, 0.17), pants, 0, 0.02, 0);
  part(J.hips, G.cyl(0.235, 0.235, 0.07, 14), '#14101f', 0, 0.1, 0, { outline: false });
  part(J.torso, G.sph(look.big ? 0.33 : 0.27, 0.33, look.big ? 0.24 : 0.2), jacket, 0, 0.3, 0);
  if (!look.big) { part(J.torso, G.box(0.09, 0.5, 0.03), '#14101f', 0.0, 0.3, 0.195, { outline: false }); part
(J.torso, G.cyl(0.16, 0.2, 0.12, 12), jacket, 0, 0.6, 0); }
  else { part(J.torso, G.cyl(0.03, 0.03, 0.5, 6), '#ffd23c', 0, 0.48, 0.2, { rot: [Math.PI / 2 - 0.3, 0, Math.P
I / 2], outline: false }); }
  const head = J.head = grp(J.neck, 0, 0, 0); head.userData.mat = mat;
  part(head, G.cyl(0.08, 0.09, 0.12, 10), skin, 0, 0.03, 0);
  part(head, G.sph(0.17, 0.2, 0.18), skin, 0, 0.2, 0.01);
```

The pattern: one `toonMat` per rig → `skeleton(d, mat)` → `part(joint, G.prim(…), colour, x, y, z, opts)` on each joint → `J.head` group under `J.neck` → `J.kind` → return `J`. The game wraps `J` in a Fighter (position, yaw, HP, AI), drives the joints with the pose tables and flashes `J.mat` on hits.

K member	What it is (reuse it; do not re-create it)
`K.THREE`	three.js r160 namespace (the game's own instance)
`K.toonMat(opts)`	The cel-shaded Lambert material with vertex colours and hit-flash. One per character rig; sub-materials with a map copy its `userData.flash` and `flashC`.
`K.basicMat(opts)`	An unlit material for glow parts (opacity driven by `K.NIGHT.value`).
`K.OUT_CHAR / OUT_PROP / OUT_CITY`	The three shared outline hull materials (widths 0.0042 / 0.0030 / 0.0034).
`K.G.sph / cap / box / cyl / tor`	The primitive geometry makers. They pick HD segment counts for you.
`K.part(parent, geom, colour, x, y, z, {rot, outline, mat, shadow})`	Bakes the colour, adds the ink hull and casts shadows. The parent group must carry `userData.mat`.
`K.grp(parent, x, y, z)` · `K.skeleton(d, mat)`	Groups; the 18-joint rig: tilt, hips, torso, neck, shL/R, elL/R, haL/R, lgL/R, knL/R, ftL/R, swL/R.
`K.HERO_D / K.THUG_D`	Rig proportions (hip height, limb lengths).
`K.blank()` · `legsStance()` · `runLegs()` · `applyPose()`	Pose objects and helpers. `K.THUG_POSES` and `K.HERO_POSES` are the game's own animations.
`K.place(geom, x, y, z, ry)` · `K.makeOut()` · `K.makeAtlas(tiles)`	The static-geometry path: world placement, the merging collector (`out.solid / glow / decal / coll / bld / lamp / hook / build`) and the single canvas atlas.
`K.CITY_MAT` · `K.GLOW_MAT` · `K.LIGHT` · `K.applyLight` · `K.NIGHT` · `K.INK`	The shared city materials, the day/dusk/night presets and the ink colour.

Prop interface: example_bench.js (complete)

```
// example_bench.js — the SHAPE every delivered static prop / kit piece must follow.
// No imports. Write placed world-space geometry into `out`; the game merges it into its chunk meshes (3 draw c
alls per chunk total).
export const META = { id: 'bench_slat', kind: 'prop', footprint: [1.8, 0.6], height: 0.9 };

export function buildBench(K, out, x, z, ang = 0, opts = {}) {
  const { THREE, place } = K, wood = opts.wood || '#9a6a44', iron = opts.iron || '#2c322e';
```

```
  const P = (g, lx, ly, lz) => { g.translate(lx, ly, lz); return place(g, x, 0.16, z, ang); }; // 0.16 = kerb / pavement height
  for (const [ly, lz, rx] of [[0.45, 0.0, 0], [0.45, 0.12, 0], [0.45, -0.12, 0]]) out.solid(P(new THREE.BoxGeometry(1.8, 0.05, 0.1), 0, ly, lz), wood);
  for (const ly of [0.62, 0.78]) out.solid(P(new THREE.BoxGeometry(1.8, 0.1, 0.05), 0, ly, -0.22), wood);
  for (const s of [-1, 1]) { out.solid(P(new THREE.BoxGeometry(0.06, 0.45, 0.42), s * 0.8, 0.22, 0), iron); out.solid(P(new THREE.BoxGeometry(0.06, 0.45, 0.05), s * 0.8, 0.7, -0.22), iron); }
  // a painted back-plate via the kit atlas (draw it once per module, reuse it for every instance)
  const atlas = buildBench.atlas || (buildBench.atlas = K.makeAtlas({ plate: { w: 256, h: 64, draw(c, w, h) { c.fillStyle = '#2f5a3a'; c.fillRect(0, 0, w, h);
    c.strokeStyle = '#f2e6c8'; c.lineWidth = 4; c.strokeRect(6, 6, w - 12, h - 12); c.fillStyle = '#f2e6c8'; c.font = 'bold 30px Georgia, serif'; c.textAlign = 'center'; c.fillText('HARBOR PARK', w / 2, 43); } } }));
  out.decal(P(new THREE.PlaneGeometry(0.9, 0.2), 0, 0.7, -0.19), atlas, 'plate');
  out.coll({ x, z, r: 0.7 });
  return META;
}
```

Rules that make code drop in

- No `import`. No globals, no `window`, no DOM except canvas drawing inside texture or atlas functions. No `Math.random()` at build time (use a seeded LCG from `opts.seed`).
- Characters return the rig `J` (bosses also export `POSES` and `ATTACKS`). Static pieces write only through `out`. Vehicles return `{ root, kind, seats, collider, waterline, animate }`.
- Facing is +Z, the base sits at y = 0 and units are metres. Materials come only from `K`. Never change `toadcity_kit.js`.

4 · Task A: Baltimore rowhouse and city-block kit

The box below is the **complete copy-paste prompt** for this task. Give it to the model together with `toadcity_starter_kit.zip`. The same text is in `prompts/TASK_A_rowhouse_city_block_kit.md`.

COPY-PASTE PROMPT · TASK A

You are the environment-kit builder for Toad City Baltimore (Claude Sonnet, extended thinking on, medium effort; use high effort for the first kit piece, because it sets the pattern). Self-test every piece in viewer.html before you move on.

Context (read first)

You are making game assets for **Toad City Baltimore**, a browser open-world brawler set on Baltimore's Inner Harbor. The heroes are Harold, an elderly toad with a cane, and his wife Mavis, who has a grey afro and fights with her handbag. They clean the crime out of each district. Everything is built in code and cel-shaded with ink outlines. The game uses **three.js r160** with plain ES modules: no bundler, no image files, no model files. Every texture is drawn on a canvas at load time.

You were given the **starter kit**: `toadcity_kit.js` (the game's own materials, outline method, rig and pose code, plus the prop collector), `viewer.html` (a standalone test bench), `example_gull_thug.js` (character interface) and `example_bench.js` (prop interface). If the files did not come with this brief, rebuild them from the appendix of the PDF. Your modules must run against the kit unchanged, because the game passes the same `K` object to them.

Conventions (all units are metres):

- Y is up, X is east and Z is south (north is −Z). Facing for a character or vehicle at yaw 0 is **+Z**, and its feet or base sit at y = 0.
- Pavement top (kerb) `CURB = 0.16`. Ground floor height `GF = 4.4`, upper floors `FH = 3.4`, parapet `PAR = 1.3`. Harbour water surface `WY = −0.85`.
- City block: 24 × 24 m. Street: 10 m wide (grid period 34 m). Brick promenade on every water edge: 6 m. Render chunks are 64 m and the collision grid is 16 m.
- Character scale: Harold is about 1.6 m tall, Mavis about 2.0 m to the top of her afro, and a standard thug about 2.0 m. The bruiser is the thug rig scaled ×1.2.

Hard rules for every delivery:

1. **No `import` statements in your modules.** Everything comes in through the `K` argument. Each file has one `export const META` and one or more `export function buildXxx(K, …)`.
2. **Materials:** create them only through `K.toonMat()`, `K.basicMat()` or `K.makeAtlas()`. Outlines must use the shared `K.OUT_CHAR`, `K.OUT_PROP` or `K.OUT_CITY` hulls (which `K.part()` and `out.build()` add for you). Do not use PBR materials, post-processing, edge-detection passes or lights inside assets. Lamps go through `out.lamp()`.
3. **Textures:** draw them on canvas only. Use at most one 1024² atlas per task family and at most 256² per character map. Do not use image files, web fonts or network fetches. Text uses font stacks such as `Impact, "Arial Black", Georgia, serif`.
4. **Deterministic:** do not call `Math.random()` at build time. Use a seeded LCG (`seed = seed * 16807 % 2147483647`) taken from `opts.seed`.
5. **Do not edit `toadcity_kit.js`.** If you need something it lacks, add the helper inside your own module and say so in the README.
6. **Real signage, original characters:** storefronts, signs and props in the Inner Harbor area **may use real Baltimore businesses' names and sign styles where they really exist there**. Examples: Phillips Seafood, Barnes & Noble and Hard Rock Cafe at the Power Plant, the harbour water-taxi branding, Natty Boh-style corner-bar signs, Utz, Old Bay. Recreate the lettering, colours and sign shapes **drawn on canvas or built from geometry**; never copy or embed logo image files, photos or traced artwork. Only put a real name where that business really is or is typical (a crab house, a corner bar, the Power Plant shops); fill everything else with plausible Baltimore signage. **Characters stay original:** no real people, likenesses, real gang colours

or symbols, official mascots or film/game characters, and no brand logos on thugs or bosses. Third-party code is allowed only under MIT or CC0, credited with its URL in `LICENSES.md`.
7. Do **not** model landmark buildings (stadiums, the aquarium, the power plant, towers, the fort, the monument and so on). The in-house team builds those.
8. Use three.js r160 APIs only. The code must have no console errors in Chrome or iOS Safari.

Task A: Baltimore rowhouse and city-block kit

Goal

Build a modular kit that fills the generic 24 × 24 m city blocks around the harbour with recognisably **Baltimore** rowhouse streets. The kit covers Formstone fronts, white marble stoops, bracketed cornices, painted window screens, corner bars and corner shops, in 3- and 4-storey variants. It must merge into the game's chunk meshes, so the whole city stays at a few draw calls per 64 m chunk.

Pieces to build

1. **`buildRowhouse(K, out, atlas, x, z, ang, opts)`**: one house. Frontage 4.2–6.0 m (2 or 3 window bays), depth 12–14 m, 3 or 4 storeys (`opts.floors`). Front facade options in `opts.front`:
2. `formstone`: faux-stone cement facing in irregular blocks, grey-beige to pink-grey, drawn on the kit atlas.
3. `brick`: red or tan brick, with optional painted brick (`opts.paint`).
4. `permastone`: a smoother variant of Formstone. The house also has:
5. **A white marble stoop**: 3–5 steps, 1.3–1.5 m wide, projecting 1.4–1.8 m onto the pavement, with a slight bevel. Use near-white `#eae6dc` with a cool shadow band.
6. **A cornice**: a bracketed metal or wood cornice at the roofline, with 0.5–0.8 m projection, 4–8 brackets and a dentil band. Painted cornice colours come from the palette.
7. **Windows**: 1/1 sash with lintels and sills; some ground-floor windows get **painted window screens**, which are original folk-art scenes drawn on canvas (a red-roof cottage by a lake, a swan, a lighthouse, a sailboat). Use at least 4 designs and no copies of real artworks.
8. The door with transom and number, a basement window grille, a downspout, and a flat roof with a low parapet, a hatch and an optional rooftop deck.
9. **`buildCornerBar(K, out, atlas, x, z, ang, opts)`**: the end-of-row corner building with a chamfered corner door, a neon or painted sign: real Baltimore corner-bar styles are welcome (Natty Boh-style beer signs, "TAVERN", "CRAB HOUSE", "BAR & GRILL"), drawn on canvas, glass-block windows and an awning. The neon glows at night through `out.glow()`.
10. **`buildCornerShop(K, out, atlas, x, z, ang, opts)`**: a corner store with a big front window, a roll-up security shutter (half-open option) and a sign ("CORNER MARKET", "CARRYOUT", "BARBER", "HARDWARE", "LAUNDROMAT", "DELI"), plus real local product signs in the window (Utz, Old Bay, Natty Boh-style), drawn on canvas.
11. **`buildRowBlock(K, out, atlas, x0, z0, x1, z1, opts)`**: fills one block. Houses face all four streets and stand shoulder to shoulder, with a corner bar or shop on 1–2 corners and a 3 m rear alley or yard strip in the middle (`out.coll` closes it). It mixes fronts, heights, colours and stoops from `opts.seed`. Neighbours never repeat the same front and colour.
12. **`KIT_TILES`**: the canvas-drawn tiles (Formstone, brick, painted screens, signs, door and window details) for **one** 1024² atlas, made with `K.makeAtlas(KIT_TILES)`.

Interface (must match exactly)

File names: `src/rowkit_tiles.js`, `src/rowkit_rowhouse.js`, `src/rowkit_corner.js` and `src/rowkit_block.js`.

```
export const META = { id: 'rowhouse', kind: 'kit', footprint: [4.8, 13], floors: [3, 4] };
export function buildRowhouse(K, out, atlas, x, z, ang = 0, opts = {}) {
  // (x, z) = centre of the FRONT wall at pavement level; the front faces +Z rotated by ang. Base sits on K's CURB
(0.16).
  // Write only through: out.solid(geom, colour, outline?), out.glow(geom, colour), out.decal(geom, atlas, tileNam
e),
  //                     out.coll({x0,x1,z0,z1} | {x,z,r}), out.bld({x0,x1,z0,z1,h}), out.lamp(x,y,z), out.hook(ty
pe, data)
```

```
  return { width, depth, height };   // so the block filler can pack houses
}
```

- Geometry is placed in **world space** with `K.place(geom, x, y, z, ry)`. `out.build()` merges everything into one toon mesh, one outline hull, one glow mesh and one decal mesh.
- **Night windows:** each window pane goes through `out.glow(paneGeom, colour)`, with roughly 45% lit chosen by the seeded RNG. Lit panes use warm `#ffe08a` / `#ffbf3c`; unlit panes are not written to glow (they stay as dark glass in `out.solid`). Shop glass and neon always glow.
- `out.bld()` adds one camera-collision box per house. `out.coll()` gives the house footprint and each stoop. `out.hook('climb', {x, z, ang, top})` marks one drainpipe or ladder per block for the CLIMB action.

Performance budget

- **One house ≤ 2,500 triangles**, a corner building ≤ 3,500, and **one full block ≤ 30,000**.
- Materials: only what `out` uses (`K.CITY_MAT`, the glow material and **one** kit atlas for the whole task). A full block must build to **≤ 4 draw calls** (plus the shadow pass).
- Outline hulls only on major masses (house box, stoop, cornice, bay edges). Use `outline: false` for anything under about 6 cm (mullions, bricks, trim strips).

Style

- Use chunky, slightly exaggerated proportions: cornices a bit deeper, stoops a bit whiter and wider than in reality, to read on a phone.
- Formstone gets bold irregular block outlines drawn in a darker tone of the base (not `K.INK`) so that it does not become noise at a distance.
- Wall colours come from the building palette (red brick `#8a4a3a`, grey stone `#6f6a62`, tan brick `#9a6a44`, plum `#5a4f5c`, green-grey `#4f5a52`, sandstone `#a4907a`) plus painted-brick pastels (`#c9b48a`, `#b8a27a`, `#9ab0b8`, `#c49a9a`).

Acceptance checklist

- ☐ All five pieces exist with the exact signatures. `viewer.html?show=street` shows two facing blocks across a 10 m street with a pavement, in day and night.
- ☐ Formstone, white marble stoops, bracketed cornices, painted screens, corner bar and corner shop are all present and recognisable.
- ☐ Both 3- and 4-storey variants exist, with at least 3 front types and at least 4 painted screen designs.
- ☐ Budgets are measured: house ≤ 2.5k, corner ≤ 3.5k and block ≤ 30k triangles; a full block uses ≤ 4 draw calls; there is one atlas in total.
- ☐ Night: about 45% of windows lit (seeded, stable on rebuild), with shop glass and neon glowing.
- ☐ Colliders, camera boxes and climb hooks are written. Nothing floats and nothing sinks below `CURB`.
- ☐ Same seed gives the same block; a different seed gives a visibly different mix.
- ☐ Signs read as real Baltimore: real names and sign styles only where they really exist or are typical, all drawn on canvas or built from geometry (no logo image files). `LICENSES.md` is present.

Do NOT

- Do not model landmarks or any specific real building.
- Do not use per-house materials or per-house meshes; everything goes through `out`.
- Do not use image textures, logo files, or photos pasted as textures. Real names and sign styles must be redrawn on canvas.

Session plan

The **first piece sets the pattern**: build the 3-storey Formstone rowhouse to full quality and self-test it before anything else. Then build the other fronts and 4 storeys, then the corner bar and corner shop, then the block filler, then the previews.

Delivery and self-test

Make one zip per task: `toadcity_task<LETTER>_<short-name>_v<N>.zip`, laid out like this:

```
src/                your modules (snake_case file names, e.g. gang_harbor_rats.js, boss_raven_king.js, rowkit_ro
whouse.js)
toadcity_kit.js     the kit, byte-identical to the one you were given
viewer.html         your test bench (start from the kit's viewer.html): imports ./src/*.js, ?show=<id>, ?night=
1, ?pose=…
previews/           <asset_id>_day.png and <asset_id>_night.png at 1280x720, plus <group>_phone_390x844.png
README.md           exports, how to view, the MEASURED budget table (parts, tris, draw calls, materials), known
issues
LICENSES.md         "All original, procedural" or credits (name, URL, licence) for any MIT / CC0 source used
```

Self-test loop (run it for every asset before you move to the next):

1. Serve the folder (`python3 -m http.server 8000`) and open `viewer.html?show=<id>`. Check day, then press N for night, then P to cycle poses.
2. Read the corner readout. **Draw calls and triangles must be inside the budget** for the asset type. Count your materials too.
3. Look at the asset at gameplay distance in a 390×844 window, the iPhone portrait view. The silhouette must read at about 6–10 m away.
4. Take screenshots. Headless Chrome works with `--use-gl=swiftshader`; wait for `window.__ready === true`, then save the `previews/` PNGs.
5. The console must show zero errors. Fix the asset before you move on.

Do not send: model or image files (GLB, OBJ, FBX, PNG/JPG textures), minified code, copies of three.js, or anything with an unclear licence.

5 · Task B: Street props and vehicles

The box below is the **complete copy-paste prompt** for this task. Give it to the model together with `toadcity_starter_kit.zip`. The same text is in `prompts/TASK_B_street_props_vehicles.md`.

COPY-PASTE PROMPT · TASK B

You are the props and vehicles builder for Toad City Baltimore (Claude Sonnet, extended thinking on, medium effort; use high effort for the first prop and the first vehicle, because they set the pattern). Self-test every piece in viewer.html before you move on.

Context (read first)

You are making game assets for **Toad City Baltimore**, a browser open-world brawler set on Baltimore's Inner Harbor. The heroes are Harold, an elderly toad with a cane, and his wife Mavis, who has a grey afro and fights with her handbag. They clean the crime out of each district. Everything is built in code and cel-shaded with ink outlines. The game uses **three.js r160** with plain ES modules: no bundler, no image files, no model files. Every texture is drawn on a canvas at load time.

You were given the **starter kit**: `toadcity_kit.js` (the game's own materials, outline method, rig and pose code, plus the prop collector), `viewer.html` (a standalone test bench), `example_gull_thug.js` (character interface) and `example_bench.js` (prop interface). If the files did not come with this brief, rebuild them from the appendix of the PDF. Your modules must run against the kit unchanged, because the game passes the same `K` object to them.

Conventions (all units are metres):

- Y is up, X is east and Z is south (north is −Z). Facing for a character or vehicle at yaw 0 is **+Z**, and its feet or base sit at y = 0.
- Pavement top (kerb) `CURB = 0.16`. Ground floor height `GF = 4.4`, upper floors `FH = 3.4`, parapet `PAR = 1.3`. Harbour water surface `WY = −0.85`.
- City block: 24 × 24 m. Street: 10 m wide (grid period 34 m). Brick promenade on every water edge: 6 m. Render chunks are 64 m and the collision grid is 16 m.
- Character scale: Harold is about 1.6 m tall, Mavis about 2.0 m to the top of her afro, and a standard thug about 2.0 m. The bruiser is the thug rig scaled ×1.2.

Hard rules for every delivery:

1. **No `import` statements in your modules.** Everything comes in through the `K` argument. Each file has one `export const META` and one or more `export function buildXxx(K, …)`.
2. **Materials:** create them only through `K.toonMat()`, `K.basicMat()` or `K.makeAtlas()`. Outlines must use the shared `K.OUT_CHAR`, `K.OUT_PROP` or `K.OUT_CITY` hulls (which `K.part()` and `out.build()` add for you). Do not use PBR materials, post-processing, edge-detection passes or lights inside assets. Lamps go through `out.lamp()`.
3. **Textures:** draw them on canvas only. Use at most one 1024² atlas per task family and at most 256² per character map. Do not use image files, web fonts or network fetches. Text uses font stacks such as `Impact, "Arial Black", Georgia, serif`.
4. **Deterministic:** do not call `Math.random()` at build time. Use a seeded LCG (`seed = seed * 16807 % 2147483647`) taken from `opts.seed`.
5. **Do not edit `toadcity_kit.js`.** If you need something it lacks, add the helper inside your own module and say so in the README.
6. **Real signage, original characters:** storefronts, signs and props in the Inner Harbor area **may use real Baltimore businesses' names and sign styles where they really exist there**. Examples: Phillips Seafood, Barnes & Noble and Hard Rock Cafe at the Power Plant, the harbour water-taxi branding, Natty Boh-style corner-bar signs, Utz, Old Bay. Recreate the lettering, colours and sign shapes **drawn on canvas or built from geometry**; never copy or embed logo image files, photos or traced artwork. Only put a real name where that business really is or is typical (a crab house, a corner bar, the Power Plant shops); fill everything else with plausible Baltimore signage. **Characters stay original:** no real people, likenesses, real gang colours or symbols, official mascots or film/game characters, and no brand logos on thugs or bosses. Third-party code is allowed only under MIT or CC0, credited with its URL in `LICENSES.md`.
7. Do **not** model landmark buildings (stadiums, the aquarium, the power plant, towers, the fort, the monument and so on). The in-house team builds those.

8. Use three.js r160 APIs only. The code must have no console errors in Chrome or iOS Safari.

Task B: Street props and vehicles

Goal

Build the street-level props and vehicles that make the harbour city feel alive. Static props merge into the city mesh through `out`. Vehicles and rideables are their own small Groups, because the heroes ride and drive them (the ACT button: RIDE, DRIVE, BOARD).

Static props (merged; signature `buildXxx(K, out, x, z, ang = 0, opts = {})`)

id	Prop	Notes	Tris budget
`lamp_acorn`	Historic acorn street lamp	Dark green cast-iron post with an acorn globe; globe through `out.glow`; calls `out.lamp(x, y, z)` at the globe	≤ 350
`lamp_cobra`	Modern cobra-head lamp	For avenues; head through `out.glow`, `out.lamp`	≤ 250
`bench_park`	Park bench	Wood slats on iron ends; `out.hook('seat', …)`	≤ 250
`bench_bus`	Bus-stop bench	Baltimore bus-stop sign style, drawn on canvas	≤ 300
`trash_can`	Public trash can	Plus a variant with an attached recycling bin	≤ 250
`crab_shack`	Crab shack / steamed-crab stand	Small wooden stand (about 4 × 3 m), counter, chalkboard menu, "STEAMED CRABS" sign that glows at night (real local flavour welcome: Old Bay tins, Utz, Natty Boh-style signs, drawn on canvas), crab-pot stack	≤ 2,500
`manhole`	Manhole cover	Flush with the road (y = 0.005), with a ring decal; `out.hook('manhole', {x, z})`	≤ 120
`ladder_wall`	Wall ladder	Fixed steel ladder from the ground to a roof height `opts.top`; `out.hook('climb', {x, z, ang, top})`	≤ 300
`ladder_drop`	Fire-escape drop ladder	Bottom section only, hanging at 1.6 m; `out.hook('climb', …)`	≤ 300
`parked_car_*`	Parked cars (sedan, hatchback, pickup)	Static versions of the drivable cars; any colour from the palette	≤ 1,500

Vehicles and rideables (own Group; signature `buildXxx(K, opts = {})`)

id	Vehicle	Notes	Budget
`water_taxi`	Harbour water taxi	About 9 m covered launch, the real harbour water-taxi branding, lettering and colour scheme, recreated on canvas and in geometry (no copied logo files), bench seats, a pilot wheel. The player drives it	≤ 4,000 tris, ≤ 8 meshes
`paddle_boat`	Two-seat pedal boat	Swan-ish or generic dragon-head pedal boat for 2 riders	≤ 2,500 tris, ≤ 6 meshes
`dragon_boat`	Racing dragon boat	About 12 m long and narrow, dragon head and tail, 10 paddler benches and a drum; paddles animate	≤ 6,000 tris, ≤ 10 meshes
`car_sedan`, `car_hatch`	Drivable cars	Wheels spin and steer; doors are not needed. Fells Point set piece: the heroes lean out of both windows	≤ 2,000 tris, ≤ 7 meshes
`box_truck`	Box truck	"MOVING & STORAGE" or a real Baltimore delivery look (e.g. Utz), drawn on canvas; the roll-up rear door is optional	≤ 2,500 tris, ≤ 7 meshes
`arabber_cart`	Arabber produce cart with pony	Painted wooden cart with a canopy, bells and fruit crates, pulled by a stylised pony with a simple walk cycle in `animate`	≤ 5,000 tris, ≤ 12 meshes
`skateboard`	Skateboard	Deck, trucks and wheels; the rider stands on `seats[0]`	≤ 600 tris, ≤ 3 meshes

Vehicle return value (must match exactly):

```
export function buildWaterTaxi(K, opts = {}) {
  const out = K.makeOut();            // build the static body with the same out.* calls as props, at the local o
rigin
  /* … out.solid(…) … */
  const root = new K.THREE.Group(); root.add(out.build());   // body = 3-4 draw calls
  const wheels = []; /* moving parts: K.grp + K.part with a shared K.toonMat (set group.userData.mat) */
  return {
    root, kind: 'boat',               // 'boat' | 'car' | 'cart' | 'board'
    seats: [{ x: 0, y: 0.9, z: -1.2, ang: 0, pose: 'stand' }],   // local-space rider anchors; pose: 'stand' | 'si
t' | 'lean_out'
    collider: { w: 2.6, l: 9.0, h: 2.4 },                        // local box, centred, along +Z
    waterline: 0.35,                  // boats: local y that sits on the water surface (WY = -0.85)
    animate(v, dt, speed, steer, t) { /* spin wheels / paddles, bob boats, pony walk; must be cheap */ },
  };
}
```

- Facing is +Z. Wheels sit on y = 0 (cars, carts, board) and boats float with `waterline` on the water plane.
- Night: headlights, tail lights, cabin lights and the crab-shack sign use `out.glow()`. Vehicles do not add lights.

Performance budget

- Static props: the per-prop triangle budgets in the table above. They add **no draw calls**, because they merge into the chunk.
- Vehicles: the triangle and mesh budgets in the table. Use at most 2 materials per vehicle (the `out` materials plus one shared `toonMat` for moving parts), and no textures beyond the shared prop atlas.
- Use **one 1024² atlas for the whole of Task B** (`src/props_tiles.js` exporting `PROP_TILES`) for signs, menus, livery text and the manhole ring.

Style

- Use chunky toy-like proportions with readable silhouettes at 10–20 m: slightly oversized lamp heads, wheels and cabins.
- Ink outlines on major masses only. Glass is a dark blue-grey (`#6a8a9a` by day) with no transparency.
- Real Baltimore flavour: real names and sign styles where they really exist (water-taxi branding, Old Bay, Utz, Natty Boh-style signs), arabber colour schemes (painted red, yellow and green wood with scrollwork), the crab-shack chalkboard, harbour-boat bunting.

Acceptance checklist

- ☐ Every prop and vehicle in the two tables exists with the exact signature and return shape.
- ☐ `viewer.html?show=props` shows all static props in a row on a pavement strip; `?show=vehicles` shows every vehicle with `animate()` running (wheels spin, paddles stroke, the pony walks, boats bob on a water plane at `WY`).
- ☐ Budgets are measured and written in the README (triangles per prop; triangles, meshes and draw calls per vehicle). There is one atlas in total.
- ☐ Hooks are present: manhole → `'manhole'`, ladders → `'climb'`, benches → `'seat'`, lamps → `out.lamp`.
- ☐ Night previews show the lamp globes, headlights and signs glowing and readable.
- ☐ Seats line up with the hero rig (Harold is about 1.6 m tall), and the `collider` box matches the visible hull.
- ☐ Liveries and signs use real Baltimore names and styles where they really exist, all redrawn on canvas or built from geometry (no logo image files). `LICENSES.md` is present.

Do NOT

- Do not embed logo image files or photos; recreate real lettering and liveries on canvas or in geometry.
- Do not model landmark buildings or piers. The in-house team does those.
- Do not use skinned meshes or GLTF. Do not add per-prop materials.

Session plan

Do the lamp and bench first (the pattern piece: self-test it, then the hooks), then the other static props, then the car (the vehicle pattern piece), then the boats, then the cart and the skateboard, then the previews.

Delivery and self-test

Make one zip per task: `toadcity_task<LETTER>_<short-name>_v<N>.zip`, laid out like this:

```
src/                your modules (snake_case file names, e.g. gang_harbor_rats.js, boss_raven_king.js, rowkit_ro
whouse.js)
toadcity_kit.js     the kit, byte-identical to the one you were given
viewer.html         your test bench (start from the kit's viewer.html): imports ./src/*.js, ?show=<id>, ?night=
1, ?pose=…
previews/           <asset_id>_day.png and <asset_id>_night.png at 1280x720, plus <group>_phone_390x844.png
README.md           exports, how to view, the MEASURED budget table (parts, tris, draw calls, materials), known
issues
LICENSES.md         "All original, procedural" or credits (name, URL, licence) for any MIT / CC0 source used
```

Self-test loop (run it for every asset before you move to the next):

1. Serve the folder (`python3 -m http.server 8000`) and open `viewer.html?show=<id>`. Check day, then press N for night, then P to cycle poses.
2. Read the corner readout. **Draw calls and triangles must be inside the budget** for the asset type. Count your materials too.
3. Look at the asset at gameplay distance in a 390×844 window, the iPhone portrait view. The silhouette must read at about 6–10 m away.
4. Take screenshots. Headless Chrome works with `--use-gl=swiftshader`; wait for `window.__ready === true`, then save the `previews/` PNGs.
5. The console must show zero errors. Fix the asset before you move on.

Do not send: model or image files (GLB, OBJ, FBX, PNG/JPG textures), minified code, copies of three.js, or anything with an unclear licence.

6 · Delivery instructions

The full delivery and self-test block is the last part of every copy-paste prompt above. In summary: **one zip per task**, named `toadcity_task<LETTER>_<short-name>_v<N>.zip`, containing `src/` (your modules), an unchanged `toadcity_kit.js`, `viewer.html`, `previews/` (day and night PNGs at 1280×720 per asset, plus a 390×844 phone framing), a `README.md` with the measured budget table, and `LICENSES.md`.

> **Licences.** Everything should be procedural or original. If any snippet comes from elsewhere, it must be MIT or CC0 and credited in `LICENSES.md` (name, URL, licence). three.js itself is MIT and is loaded by the viewer only, never copied into `src/`. Do not send anything with an unclear licence, and no photographs, logo image files or traced artwork. Real sign wording and styles are fine when redrawn in code.

Hand-back: one zip per task, versioned (`_v1`, `_v2` …); re-deliver the whole zip after fixes, not a diff. A separate QA model reviews each zip (pass/fail per asset, with numbered fixes), so expect one fix round. List anything unfinished under *Known issues* in the README.

Appendix A · Kit core (props and city)

The prop and city core of `toadcity_kit.js`, printed so you can rebuild it if the zip is missing (… marks omitted sections: characters and poses, lighting presets). The file ends by exporting every function and constant above as `export const K = { … }`. Use the zip copy when you have it.

```
// toadcity_kit.js — Toad City Baltimore: the shared character / prop kit (HD cel-shaded path only).
// This is a faithful standalone copy of the game's own materials, outline method and rig code (three.js r160),
// so modules built against it drop straight into the game. Deliveries NEVER import anything: they receive this object as `K`.
import * as THREE from 'three';

const TAU = Math.PI * 2;
const clamp = (v, a, b) => (v < a ? a : v > b ? b : v);
const lerp = (a, b, t) => a + (b - a) * t;
const damp = (a, b, r, dt) => a + (b - a) * (1 - Math.exp(-r * dt));
const wrapA = (a) => { a = (a + Math.PI) % TAU; if (a < 0) a += TAU; return a - Math.PI; };
const ease = (t) => t * t * (3 - 2 * t);
const QUAL = { hd: true, outlines: [], texs: [] };   // the game runs HD by default; the kit is HD-only
const INK = '#14101f';                               // the one ink colour: outlines, shadows on signs, dark trims
const NIGHT = { value: 0 };                          // 0 day .. 1 night (the game drives it; viewers set it)

// ---- toon (cel) material: MeshLambert + vertex colours + 3 hard bands + a hit-flash uniform ----
function toonMat(opts = {}) {
  const m = new THREE.MeshLambertMaterial(Object.assign({ vertexColors: true }, opts));
  m.userData.flash = { value: 0 }; m.userData.flashC = { value: new THREE.Color(1, 1, 1) };
  m.onBeforeCompile = (sh) => {
    sh.uniforms.uFlash = m.userData.flash; sh.uniforms.uFlashC = m.userData.flashC;
    sh.fragmentShader = sh.fragmentShader
      .replace('#include <common>', '#include <common>\nuniform float uFlash; uniform vec3 uFlashC;')
      .replace('#include <opaque_fragment>', `
        { // HD cel bands: shadow 0.46 / mid 0.78 / lit 1.06 of the base colour, hard edges; shadows tinted cool
          float ld = max(dot(diffuseColor.rgb, vec3(0.299, 0.587, 0.114)), 1e-3), lo = dot(outgoingLight, vec3(0.299, 0.587, 0.114)), r = lo / ld;
          float b = mix(0.46, 0.78, smoothstep(0.38, 0.44, r)); b = mix(b, 1.06, smoothstep(0.86, 0.92, r)); b = mix(b, r, step(1.35, r));
          outgoingLight *= b / max(r, 1e-3);
          outgoingLight = mix(outgoingLight, outgoingLight * vec3(0.88, 0.86, 1.1), (1.0 - smoothstep(0.38, 0.44, r)) * 0.5); }
        outgoingLight = mix(outgoingLight, uFlashC, uFlash);
        #include <opaque_fragment>`);
  };
  m.customProgramCacheKey = () => 'psx' + (opts.map ? 'M' : '');
  return m;
}
// unlit material for neon, glows and lamp heads (night-only things fade with opacity)
function basicMat(opts) { return new THREE.MeshBasicMaterial(opts); }
// ---- ink outlines: inverted hulls. Back faces pushed out along a mitred direction (attribute `odir`), width ~constant on screen ----
const OUT_U = { fogC: { value: new THREE.Color('#000') }, fogN: { value: 100 }, fogF: { value: 200 } };
function outlineMat(w) {
  return new THREE.ShaderMaterial({
    uniforms: { uW: { value: w }, uC: { value: new THREE.Color(INK) }, uFogC: OUT_U.fogC, uFogN: OUT_U.fogN, uFogF: OUT_U.fogF },
    vertexShader: `attribute vec3 odir; uniform float uW; varying float vD;
      void main() { vec4 wp = modelMatrix * vec4(position, 1.0); vec3 wd = mat3(modelMatrix) * odir;
        float d = distance(wp.xyz, cameraPosition); wp.xyz += wd * uW * clamp(d, 2.5, 36.0);
        vD = d; gl_Position = projectionMatrix * viewMatrix * wp; }`,
    fragmentShader: `uniform vec3 uC; uniform vec3 uFogC; uniform float uFogN; uniform float uFogF; varying float vD;
      void main() { gl_FragColor = vec4(mix(uC, uFogC, smoothstep(uFogN, uFogF, vD)), 1.0);
        #include <colorspace_fragment>
      }`,
    side: THREE.BackSide,
  });
}
// the three shared outline widths: city 0.0034, characters 0.0042, props 0.0030 (fraction of camera distance, clamped 2.5..36 m)
const OUT_CITY = outlineMat(0.0034), OUT_CHAR = outlineMat(0.0042), OUT_PROP = outlineMat(0.0030);
function addOdir(g) {
  const p = g.attributes.position, n = g.attributes.normal, map = new Map(), keys = [];
  for (let i = 0; i < p.count; i++) {
    const k = Math.round(p.getX(i) * 1e3) + ',' + Math.round(p.getY(i) * 1e3) + ',' + Math.round(p.getZ(i) * 1e3);
    keys.push(k); let e = map.get(k); if (!e) map.set(k, (e = []));
    const nx = n.getX(i), ny = n.getY(i), nz = n.getZ(i);
    if (!e.some((q) => q[0] * nx + q[1] * ny + q[2] * nz > 0.985)) e.push([nx, ny, nz]);
  }
  const res = new Map();
  for (const [k, e] of map) {
    let x = 0, y = 0, z = 0; for (const q of e) { x += q[0]; y += q[1]; z += q[2]; }
    const l = Math.hypot(x, y, z) || 1; x /= l; y /= l; z /= l;
    let mn = 1; for (const q of e) mn = Math.min(mn, q[0] * x + q[1] * y + q[2] * z);
    const s = 1 / Math.max(mn, 0.35); res.set(k, [x * s, y * s, z * s]);
  }
  const o = new Float32Array(p.count * 3);
  for (let i = 0; i < p.count; i++) { const d = res.get(keys[i]); o[i * 3] = d[0]; o[i * 3 + 1] = d[1]; o[i * 3 + 2] = d[2]; }
  g.setAttribute('odir', new THREE.BufferAttribute(o, 3));
  return g;
}
function bakeColor(g, col) {
  const c = new THREE.Color(col), n = g.attributes.position.count, a = new Float32Array(n * 3);
  for (let i = 0; i < n; i++) { a[i * 3] = c.r; a[i * 3 + 1] = c.g; a[i * 3 + 2] = c.b; }
  g.setAttribute('color', new THREE.BufferAttribute(a, 3)); return g;
}
// canvas -> texture (the game draws every texture on a canvas at load: no image files). HD = full-size source, trilinear mipmaps.
function lowTex(src) {
  const t = new THREE.CanvasTexture(src); t.colorSpace = THREE.SRGBColorSpace; t.minFilter = THREE.LinearMipmapLinearFilter; t.magFilter = THREE.Line
arFilter;
  t.generateMipmaps = true; t.anisotropy = 4; t.userData.src = src; QUAL.texs.push(t); return t;
}

// …

// ---------------- characters: procedural stylised rigs (groups of toon parts with outline hulls) ----------------
const G = { // low-poly on purpose (32-bit era budgets)
```

```
  sph: (rx, ry, rz, ws = 8, hs = 6) => new THREE.SphereGeometry(1, QUAL.hd ? Math.max(ws * 2, 18) : Math.min(ws, 9), QUAL.hd ? Math.max(hs * 2, 12) :
Math.min(hs, 7)).scale(rx, ry, rz),
  cap: (r, len, rs = 6) => new THREE.CapsuleGeometry(r, len, QUAL.hd ? 4 : 2, QUAL.hd ? 12 : Math.min(rs, 6)),
  box: (w, h, d) => new THREE.BoxGeometry(w, h, d),
  cyl: (rt, rb, h, s = 7, open = false) => new THREE.CylinderGeometry(rt, rb, h, QUAL.hd ? Math.max(s * 2, 14) : Math.min(s, 8), 1, open),
  tor: (R, r, arc = TAU) => new THREE.TorusGeometry(R, r, QUAL.hd ? 6 : 3, QUAL.hd ? 16 : 7, arc),
};
function part(parent, geom, col, x = 0, y = 0, z = 0, o = {}) {
  bakeColor(geom, col); const ol = o.outline !== false && geom.attributes.normal;
  if (ol) addOdir(geom);
  const m = new THREE.Mesh(geom, o.mat || parent.userData.mat || parent.parent?.userData.mat);
  m.position.set(x, y, z); if (o.rot) m.rotation.set(o.rot[0], o.rot[1], o.rot[2]);
  if (ol) { const h = new THREE.Mesh(geom, o.olMat || OUT_CHAR); h.castShadow = false; h.visible = QUAL.hd; m.add(h); QUAL.outlines.push(h); }
  geom.computeBoundingSphere(); m.castShadow = o.shadow !== false && geom.boundingSphere.radius > 0.1; parent.add(m);
  return m;
}
function grp(parent, x = 0, y = 0, z = 0) { const g = new THREE.Group(); g.position.set(x, y, z); parent.add(g); return g; }

// …
// ================ static props / building kit: merged geometry =================
const _m4 = new THREE.Matrix4(), _q = new THREE.Quaternion(), _e = new THREE.Euler(), _v = new THREE.Vector3(), _s = new THREE.Vector3();
// place(): bake a world transform into a geometry (the city is built in world space, then merged)
function place(g, x, y, z, ry = 0, rx = 0, rz = 0) { return g.applyMatrix4(_m4.compose(_v.set(x, y, z), _q.setFromEuler(_e.set(rx, ry, rz)), _s.set
(1, 1, 1))); }
// makeOut(): the collector a static kit piece writes into. In the game the same five calls feed the chunked city meshes
// (one merged toon mesh + one merged outline hull per 64 m chunk), the night-glow layer and the collision grid.
//   out.solid(geom, colour, outline = true)   opaque cel-shaded geometry (already placed in world space)
//   out.glow(geom, colour)                    unlit geometry that lights up at night (window panes, sign faces, lamp heads); dark by day
//   out.coll({x, z, r} | {x0, x1, z0, z1})    character collider (circle or axis-aligned box), metres
//   out.bld({x0, x1, z0, z1, h})              camera-blocking box + roof height
//   out.lamp(x, y, z)                         a street-lamp light point (the game adds the glow pool, halo and pooled point light)
//   out.hook(type, data)                      a gameplay hook the ACT button uses: 'manhole' {x, z} (drops to the sewer), 'climb' {x, z, ang, top}
//                                             (ladder / climbable wall, top = roof height), 'seat' {x, y, z, ang} (bench sit spot)
//   out.decal(geom, atlas, name)              textured geometry (painted window screens, signs, Formstone pattern): geom's 0..1 uv is remapped
//                                             into the named tile of a kit atlas made with makeAtlas(); alpha < 0.5 is cut out
function mergeGeoms(list, withUv = false) {
  let n = 0; for (const g of list) n += g.attributes.position.count;
  const P = new Float32Array(n * 3), N = new Float32Array(n * 3), C = new Float32Array(n * 3), U = withUv ? new Float32Array(n * 2) : null; let o =
0;
  for (const g of list) { const c = g.attributes.position.count; P.set(g.attributes.position.array, o * 3); N.set(g.attributes.normal.array, o * 3);
C.set(g.attributes.color.array, o * 3); if (U) U.set(g.attributes.uv.array, o * 2); o += c; }
  const m = new THREE.BufferGeometry(); m.setAttribute('position', new THREE.BufferAttribute(P, 3)); m.setAttribute('normal', new THREE.BufferAttribu
te(N, 3)); m.setAttribute('color', new THREE.BufferAttribute(C, 3));
  if (U) m.setAttribute('uv', new THREE.BufferAttribute(U, 2));
  m.computeBoundingSphere(); return m;
}
const CITY_MAT = toonMat();                                  // ONE shared material for every static kit piece
const GLOW_MAT = new THREE.MeshBasicMaterial({ vertexColors: true, transparent: true, opacity: 0 });
// makeAtlas(): every texture a kit needs is DRAWN on canvas at load (no image files) and packed into ONE 1024x1024 atlas
//   tiles = { name: { w, h, draw(ctx, w, h) } }   (w, h in px; keep the total under 1024x1024)
function makeAtlas(tiles, size = 1024) {
  const c = document.createElement('canvas'); c.width = c.height = size; const x = c.getContext('2d'), R = {}; let ax = 0, ay = 0, row = 0;
  for (const [name, t] of Object.entries(tiles)) {
    if (ax + t.w > size) { ax = 0; ay += row; row = 0; } if (ay + t.h > size) throw new Error('atlas full at ' + name);
    x.save(); x.translate(ax, ay); x.beginPath(); x.rect(0, 0, t.w, t.h); x.clip(); t.draw(x, t.w, t.h); x.restore();
    R[name] = [(ax + 1) / size, 1 - (ay + t.h - 1) / size, (t.w - 2) / size, (t.h - 2) / size]; ax += t.w; row = Math.max(row, t.h);
  }
  const tex = lowTex(c); const mat = toonMat({ map: tex, alphaTest: 0.5, side: THREE.DoubleSide });
  return { canvas: c, tex, mat, rect: (n) => R[n] };
}
function makeOut() {
  const S = [], OL = [], GL = [], DC = new Map(), out = { COLL: [], BLD: [], LAMPS: [], HOOKS: {} };
  const flat = (g) => (g.index ? g.toNonIndexed() : g);
  out.solid = (g, col, outline = true) => { g = flat(g); bakeColor(g, col); if (g.attributes.uv) g.deleteAttribute('uv'); S.push(g); if (outline) OL.
push(g); };
  out.glow = (g, col) => { g = flat(g); bakeColor(g, col); if (g.attributes.uv) g.deleteAttribute('uv'); GL.push(g); };
  out.decal = (g, atlas, name) => { g = flat(g); const r = atlas.rect(name), uv = g.attributes.uv; if (!r) throw new Error('no tile ' + name);
    for (let i = 0; i < uv.count; i++) uv.setXY(i, r[0] + uv.getX(i) * r[2], r[1] + uv.getY(i) * r[3]);
    bakeColor(g, '#ffffff'); (DC.get(atlas) || DC.set(atlas, []).get(atlas)).push(g); };
  out.coll = (c) => out.COLL.push(c); out.bld = (b) => out.BLD.push(b); out.lamp = (x, y, z) => out.LAMPS.push({ x, y, z });
  out.hook = (type, data) => (out.HOOKS[type] || (out.HOOKS[type] = [])).push(data);
  // build(): 1 toon mesh + 1 outline hull + 1 glow mesh = 3 draw calls for everything written so far
  out.build = () => {
    const grp = new THREE.Group();
    if (S.length) { const m = new THREE.Mesh(mergeGeoms(S), CITY_MAT); m.castShadow = m.receiveShadow = true; grp.add(m); }
    if (OL.length) { const g = addOdir(mergeGeoms(OL)); const h = new THREE.Mesh(g, OUT_CITY); grp.add(h); QUAL.outlines.push(h); }
    if (GL.length) { const m = new THREE.Mesh(mergeGeoms(GL), GLOW_MAT); m.renderOrder = 2; grp.add(m); }
    for (const [atlas, list] of DC) { const m = new THREE.Mesh(mergeGeoms(list, true), atlas.mat); m.castShadow = m.receiveShadow = true; grp.add(m);
} // +1 call per atlas
    return grp;
  };
  return out;
}
```

Appendix B · Haiku helper prompts

Short, low-effort prompts for **Claude Haiku** (extended thinking off). Use them only on Sonnet modules that have **passed QA**. Haiku makes colour, window, sign and trim variations and tidies file names and READMEs, without touching geometry logic, budgets or interfaces. The same text is in `prompts/HAIKU_helper_prompts.md`.

H1: Colour variants

```
Here is a finished Toad City module (three.js, no imports, uses K). Add an exported const COLOR_VARIANTS = [ …
] with
6 colour sets for the colour options it already reads from opts (do not add new options). Use ONLY these hexes:
#8a4a3a #6f6a62 #9a6a44 #5a4f5c #4f5a52 #a4907a #c9b48a #b8a27a #9ab0b8 #c49a9a #2f5a3a #2c322e #eae6dc.
Never use #000000 or #ffffff. Change nothing else. Return the full file.
```

H2: Window variants

```
In this rowhouse module, add 3 extra window styles to the existing window option list (e.g. 2/2 sash, arched li
ntel,
shuttered). Reuse the existing window function and its out.solid / out.glow calls; keep each new style within t
he
same triangle count as the current window (±10%). Return only the changed functions and the new option names.
```

H3: Sign text variants

```
This module draws shop/bar signs on canvas tiles. Add 8 more sign texts that fit Baltimore corner businesses
(e.g. "CARRYOUT", "SUBS & PIZZA", "LIQUORS", "BAIT & TACKLE", Natty Boh-style beer signs, Utz and Old Bay windo
w
signs). Real Baltimore names and sign styles are allowed; redraw them on canvas, never paste logo image files.
Keep the same font stack, colours and tile size. Return the updated tile list.
```

H4: Trim and cornice variants

```
Add 4 cornice / trim colour + bracket-count presets to this module's opts (bracket counts 4–8, colours from the
palette above). Keep geometry code unchanged; only add presets and a one-line comment each. Return the full fil
e.
```

H5: Tidy file names and README

```
Rename files in this delivery to snake_case with the task prefix (rowkit_, props_, veh_), update every referenc
e in
viewer.html and README.md, and rewrite README.md to this outline: 1) Exports (name, signature, one line each)
2) How to view (viewer.html URLs) 3) Measured budget table (copy numbers, do not invent) 4) Variants 5) Known i
ssues.
Do not change any code inside the modules. List every rename you made.
```

H6: Licence check

```
Read these files. List every external source, URL, or copied snippet you can find. If there are none, write
LICENSES.md as "All assets are original and procedural (three.js r160, MIT, loaded by the viewer only)."
If there are some, list name, URL, licence; flag anything that is not MIT or CC0.
```